Identifying and Categorizing Challenges in Large-Scale Agile Software Development Projects: Insights from Two Swedish Companies

Hina Saeeda
Department of Computer Science, Karlstad University
Universitetsgatan 2, 651 88
Karlstad, Sweden
Hina.saeeda@kau.se

Muhammad Ovais Ahmad
Department of Computer Science, Karlstad University
Universitetsgatan 2, 651 88
Karlstad, Sweden
Ovais.Ahmad@kau.se

Tomas Gustavsson
Department of Computer Science, Karlstad University
Universitetsgatan 2, 651 88
Karlstad, Sweden
tomas.gustavsson@kau.se

ABSTRACT

We conducted a case study to examine the challenges encountered in large-scale agile development (LSAD) within two Swedish software companies. While agile methodologies have proven successful in small and medium-sized projects, their implementation in large-scale software development projects can be problematic. To identify these challenges, we employed thematic analysis, which revealed a total of 26 distinct challenges. These challenges were categorized into three main themes: Processes and practices, Teams, and Organizational-level challenges in LSAD. By recognizing and addressing these challenges, projects operating in similar contexts can synchronize their activities and harness the advantages of agile methodologies at a large scale. The article delves into comprehensive discussions on these challenges, offering valuable insights and directions for future research endeavors.

CCS Concepts

•**Software Engineering** → **Software Process Improvement;**

Keywords

Large-scale Agile, Agile software development, Software development, empirical studies, software companies

1. INTRODUCTION

Since introducing the "Agile Manifesto" agile methodologies have gained much popularity and success in software development [1]. Agile methods intend to solve the persistent problems of traditional development by taking an iterative and incremental approach to software development. The agile approach emerged as a solution to software development problems, including exceeding budgets, poor code quality, and missed deadlines experienced by traditional software development methods [2]. Agile methods evolve requirements and solutions through collaboration between self-organizing and cross-functional teams [29] [4].

Agile methods were designed for small-scale projects with self-organizing teams. However, the potential benefits of agile methods have made them appealing to large projects and companies [11][22]. Because agile methods tend to scale (can scale with certain limitations), many companies are switching to an agile way of working on large projects. Companies are moving towards large-scale agile to enhance the quality of the software and maximize the value they provide to customers [15].

Agile significantly improved large-scale projects by emphasizing change tolerance, continuous delivery, and collaboration across teams and clients [6]. However, implementing an agile way of working in large-scale projects is more complex [11]. Irrespective of LSAD challenges, practitioners want to adopt agile methods to large projects to follow the fast-paced nature of the software industry [10].

Large-scale agile challenges and success factors are currently the subjects of active research. Such as Dikert et al. [11] reported 9 categories of challenges faced in large-scale agile transformation, such as communication and coordination challenges in multi-team environments, agile understanding and implementation issues, requirements engineering and quality assurance challenges, and so on. Larger companies typically have fewer interactions with their customers than smaller ones, making the customer unsatisfied at the time of delivery [26]. Kalenda et al. [19] [3] highlighted that resistance to change and integration with non-agile business processes are critical challenges in agile scaling.

Despite the growing research on large-scale agile software development, existing studies lack empirical evidence and require further investigations [11] [19] [12] [15] [3]. Most case studies focus on agile transition challenges in large-scale projects [9]. There is a need for empirical studies mainly focusing on LASD challenges. Therefore, the goal of our research is to investigate challenges in LASD projects rather than challenges in agile transformation. We aim to answer the following research question:

What are the challenges in large-scale agile software development projects?

Copyright is held by the authors. This work is based on an earlier work: Challenges in Large-Scale Agile Software Development Projects, in The 38th ACM/SIGAPP Symposium on Applied Computing (SAC '23), March 27-April 2, 2023, Tallinn, Estonia, ISBN 978-1-4503-9517-5/23/03 ©ACM, 2023. http://dx.doi.org/10.1145/3555776.3577662

As clear from our research question, our main focus is to understand the challenges in large-scale agile software development projects rather than on agile scaling challenges in large-size software development projects or agile transformation. We conducted a case study involving two large multinational companies following agile methods. We performed a set of semi-structured interviews to collect data to address the research question. Our study provides empirical evidence on large-scale agile software development challenges to extend the body of knowledge. Our research investigation scope was limited to LSAD processes and practice challenges in the previous work. In the current version, we are extending the scope of our study to include challenges in LSAD teams and organizational-level challenges in LSAD as well.

The rest of the study is structured as follows: The related work is presented in Section 2. The research method is explained in Section 3. Section 4 discusses the results, and Section 5 examines threats to validity and limitations. Section 6 presents the discussion and conclusion.

2. RELATED WORK

Agile software development is a collection of iterative and incremental development methods based on the Agile Manifesto values and principles [7]. Although agile methods are mostly rebranded from existing good software development methods, they are considered an alternative to traditional ones. Traditional methods focus on up-front planning and strict change management, while agile methods accept and manage change efficiently [11]. Researchers have shown that accommodating change may play a role in both success and failure of agile methods. Agile methods are recognized in the literature for facilitating early time to market, client feedback, and early risk mitigation [13]. Although agile methods have enhanced the satisfaction of both clients and developers, working on large-scale agile projects is challenging [14].

The existing literature identifies the interpretation of agile at scale in terms of i) the number of people in the organization, ii) the size of the teams, and iii) the number of teams involved in the development process. Dingsoyr et al.[12, p. 275] measured large scale through the existence of multiple collaborative teams as *"two to nine collaborating teams and a very large scale over ten collaborating teams."* According to Dikert et al. [11, p. 88] *"large-scale denote software development companies with 50 or more people or at least six teams".*

Dybå et al. [13] conducted a comprehensive review of empirical studies on agile software development to identify the characteristics and practices of agile software development and the benefits and limitations of using this approach. They found that agile software development is characterized by close collaboration between developers and customers, iterative and incremental development, and a focus on delivering working software quickly. The review also identified several practices commonly used in agile development, including test-driven development, continuous integration, and pair programming. The review found that agile software development has several potential benefits, including increased customer satisfaction, improved software quality, and better team performance. However, Dybå et al. [13] also identified several limitations and challenges associated with using agile development, such as the need for experienced developers, the difficulty of scaling agile practices to large projects, and the challenge of integrating agile methods with traditional software development processes. It is further explained that companies with multiple teams face challenges adopting agile methods because of interdependencies between projects and teams.

Hobbs and Petit et al. [18] found that agile methods have become increasingly popular on large projects in large organizations, as they provide a flexible and adaptive approach to managing complex projects. However, they also noted that adopting agile methods in this context could be challenging, as large organizations often have complex and hierarchical structures that may not be suitable for agile methods. They further reported that larger companies and projects face unique organizational challenges since they coordinate with multiple development teams, contact outside business analysts, and integrate existing systems. The study identified several key factors that can facilitate the successful adoption of agile methods on large projects in large organizations, including strong executive support, a culture of collaboration and openness, and effective communication and coordination among project teams. The authors also emphasized the importance of tailoring agile methods to the specific needs and characteristics of large organizations and their projects.

Dikert et al. [11] systematically reviewed large-scale agile transformation literature, examined 52 articles, and identified 21 challenges and 18 success factors. The identified challenges and success factors are categorized into four dimensions: organization, culture, methodology, and infrastructure. The most significant challenges were related to organizational and cultural factors, such as resistance to change, lack of leadership support, and the need for cultural change. In contrast, the most significant success factors were related to methodology and infrastructure, such as using agile project management tools and techniques, effective communication, and team collaboration. It was also reported that the success of large-scale agile transformations depends on the interplay of different factors across the four dimensions and that there is no one-size-fits-all solution. They conclude that successful large-scale agile transformations require a holistic approach that addresses the challenges and leverages the success factors across all dimensions.

Kalenda et al. [19] comprehensively analyze the practices, challenges, and success factors associated with scaling agile in large organizations. They conducted a multiple-case study involving five large organizations with 6,000 employees and analyzed the data using a grounded theory approach. The study identified several effective practices in scaling agile in large organizations, such as creating cross-functional teams, establishing agile governance structures, and adopting agile methodologies tailored to the organization's needs. Further challenges were identified that organizations face when scaling agile, such as resistance to change, difficulties in communication and collaboration, an overly aggressive roll-out timeframe, quality assurance concerns, integration into existing non-agile business processes, and issues with

integrating agile with other business processes. Finally, the study identified several success factors to help organizations overcome these challenges and successfully scale agile. These factors include strong leadership and support from top management, effective communication and collaboration across teams, and a commitment to continuous improvement and learning.

Kischelewski and Richter et al. [21] discuss the challenges and success factors in implementing large-scale agile methods in organizations. They conducted a qualitative study involving 14 semi-structured interviews with experienced agile practitioners from different organizations. The study identified several challenges while implementing large-scale agile methods, such as organizational resistance to change, inadequate training and education, communication barriers, a lack of clear goals, and insufficient management support. They identified that most challenges in large-scale agile are related to communication, coordination, and knowledge management. They also proposed a framework for implementing large-scale agile, which consists of six stages: preparation, initiation, education and training, pilot project, roll-out, and continuous improvement. The study concludes by highlighting the need for further research to investigate the effectiveness of this framework in different organizational contexts.

Edison et al. [15] compare the large-scale agile methods: SAFe, LeSS, Scrum-at-Scale, DAD, and the Spotify model. In total, 191 primary studies across 134 case companies were identified. This study uncovers 31 challenges grouped into 9 major categories. The major challenges discovered are inter-team coordination, organizational structure, architecture, requirements engineering, customer collaboration, method adoption, change measurement, team-related issues, and project management issues.

Sithambaram et al. [31]discussed the growing importance of the need for project managers to address the unique challenges that can arise in large-scale agile software development projects. The article identifies and explores several key challenges, including communication barriers, a lack of stakeholder involvement, resistance to change, and insufficient training and knowledge of agile teams and managers. Further, some recommendations for addressing these challenges and improving the success of agile projects are discussed. These recommendations are developing clear communication channels, involving stakeholders, implementing change management strategies, and providing adequate training and support for project team members. The article also highlights the importance of effective project management practices and emphasizes the need for project managers to address the challenges and complexities of agile projects proactively.

Shuib and Rosnani et al. [30] provide a scientific and insightful analysis of the challenges and opportunities of adopting software quality assurance practices in agile development methodology. They discuss the principles of agile development methodology and the importance of software quality assurance practices. They highlight the challenges of ensuring software quality in agile development due to its focus on flexibility and speed. They argue that software quality assurance practices should be integrated into agile development projects to address these challenges. They propose several strategies for integrating quality assurance practices into agile development projects, including test-driven development, continuous integration, code reviews, and automated testing. The authors further discuss the benefits and limitations of each approach and provide practical recommendations for their implementation. They also emphasize the need for a cultural shift in agile development teams to prioritize software quality and for project managers to provide adequate training and resources to support quality assurance practices. The author argues that adopting software quality assurance practices in agile development can lead to improved software quality, higher customer satisfaction, and greater team productivity.

Steghofer, Jan-Philipp and Knauss et al. [32] explore organizations' challenges when scaling agile development practices for safety-critical systems. While agile development has been widely adopted in many industries, including software development, it is less prevalent in safety-critical system development due to the strict regulatory requirements and safety standards. The authors argue that scaling agile development practices for safety-critical systems is particularly challenging due to the complexity of the systems, the need for rigorous testing and verification, and compliance with safety standards and regulations. Further, several key challenges associated with scaling agile development practices for safety-critical systems, including the need to ensure the safety of the system while maintaining the agility and flexibility of the development process, the need to manage dependencies and coordination across multiple teams and stakeholders, and the need to address regulatory requirements and compliance issues, are discussed in detail. In addition to the identification of the challenges, several solutions to address these challenges are discussed, including the use of agile techniques such as continuous integration and continuous delivery, the adoption of safety standards and best practices, and the development of cross-functional teams with expertise in both agile development and safety-critical systems.

Stray and Moe et al. [33] conducted a case study to explore dependency management challenges in large-scale agile projects, particularly in DevOps teams. They identified communication gaps, conflicting priorities, and technical dependencies as the major challenges that impact dependency management in large-scale agile software development projects.

After thoroughly investigating and identifying the existing challenges, they propose a framework for managing dependencies in large-scale agile projects, including key practices such as dependency mapping, prioritization, and ownership. They also suggest using tools and techniques such as visualizations and automated testing to support these practices. They conducted a case study on a large-scale Agile project involving several DevOps teams to evaluate the proposed framework. The case study provides empirical evidence of the effectiveness of the proposed framework, demonstrating improvements in communication, alignment, and overall project outcomes. The authors also discuss the framework's limitations and suggest directions for future research. One limitation is that the framework may not apply to all

projects or contexts. Another is that the framework assumes all dependencies can be identified and mapped in advance. The authors suggest exploring the framework's effectiveness in different contexts and investigating techniques for managing dependencies in complex projects. Further research is needed to develop more effective approaches for managing dependencies in large-scale Agile projects.

The research gaps related to large-scale agile software development challenges identified by [12, 15, 19, 21, 31, 30, 32, 33] serve as the foundation for the research question we address in our study.

3. RESEARCH METHOD

We conducted a case study [5] on teams from two Swedish software companies working on large-scale agile projects.

3.1 Companies Context

CaseH, a car rental company. It is one of the three largest car rental automobile-holding corporations with a 36 percent market share, operating in 160 countries and employing over 23,000 people. The company is working on different technologies to improve car rental services (i.e., introducing satellite radio into rental fleets, hourly car rental calculations, a photosystem to record damage to rental cars, and large-scale car rental kiosks, etc.). The company started implementing SCRUM in 2017 with the help of a consultancy company. In 2018, they realized the need for team coordination and implemented large-scale practices. Specifically, CaseH started to perform SCRUM of SCRUM (SoS) practices and Product Owner Synchronization (POSync) with different stakeholders. The number of teams has changed, growing to 8 teams with multiple stakeholders in the Nordic countries. Each team consists of 5 to 10 members. In total, the development department consists of over 100 people. We interview six participants from CaseH (PH6 to PH11) in different development teams working on different system modules, including invoicing, booking, damage costing, damage handling, car replacement, etc.

CaseJ is a retail chain company selling products to villa owners, home improvement workers, farmers, and craftsmen. Sales are carried out online and in more than 120 department stores in Sweden, Norway, Poland, and Finland, employing around 4,000 people. To maintain sales globally on the web, the software development teams worked on different web management modules such as web system management, digital e-commerce platform management, product and service registration, API and CMS setup, warehouse management, etc. As the organization grew, CaseJ started implementing SCRUM in 2016. The number of teams has grown and now consists of 18 teams with 2 to 10 members involving about 150 people. We interview five participants from CaseJ (PJ1 to PJ5) working on different system modules.

3.2 Data Collection

Eleven semi-structured interviews were conducted with participants from both Case companies from May to July 2022. (as shown in Table 1). All interviews were performed online on Zoom, each lasting 1-2 hours. One researcher who participated in the interviews checked each transcription by listening to the recording and correcting possible transcription errors. After removing the errors, the interview transcripts were data cleaned and organized question by question by the second researcher. All interviews were anonymized to protect the privacy of the involved companies and related participants. The finalized, cleaned, and organized interviews were returned to the companies for validation.

The interviewees were from diverse roles: software developers, team leads, product owners, product managers, SCRUM masters, and testers/QA personnel. Among these participants, four have more than fifteen years of experience, while three out of the eleven have more than twenty years of working experience. Whereas two members have more than five years of experience, the other two are relatively new to the field, with only one year of experience. All of the members had practical experience with agile methods. We used purposeful sampling [24] and selected software development teams with at least one year of work experience using agile methods. The benefit of purposeful sampling is to allow the researcher to understand the phenomenon from the selected experienced participants. The interview questions were based on these main sections: (a) provide an introduction to the study goal and reporting; (b) provide interviewees' background information; (c) provide product/project background (d) adopt software development processes and practices.

3.3 Data Analysis

All the interviews were analyzed with thematic analysis. Thematic analysis is a way of seeing themes and patterns from the collected information [16]. One researcher performed line-by-line coding of the data and recorded concepts regarding software processes and practice challenges in large-scale agile development. To develop different descriptive themes, all three researchers further analyze the inductively coded concepts at a higher abstraction level. The finalized themes helped to transform and group the multiple codes into a smaller analytical unit. The codes were grouped under the best-matching themes based on the found interrelationships (see Table: 3 for details).

Table 1: Summary of case study participants

Interviewee's Title	#	Company
Software Developer	4	CaseH(PH6,PH7,PH8), CaseJ (PJ1)
SCRUM master	2	CaseH (PH11), CaseJ (PJ2)
Tester/QA	2	CaseH (PH9,PH10)
Team lead	1	(PJ3)
Product owner	1	CaseJ (PJ4)
Project Manager	1	CaseJ (P5)

Both of the chosen cases companies are implementing agile methods. Table 2 summarizes the Agile practices used by CaseJ and CaseH. In both CaseJ and CaseH, the SCRUM master role is rotated between team members and has no dedicated product owner role in their teams. A lack of continuity in refinement meetings leaves team members somewhat directionless*(refinement meetings help teams arrive at clearly-defined and prioritized blocks of work that can be*

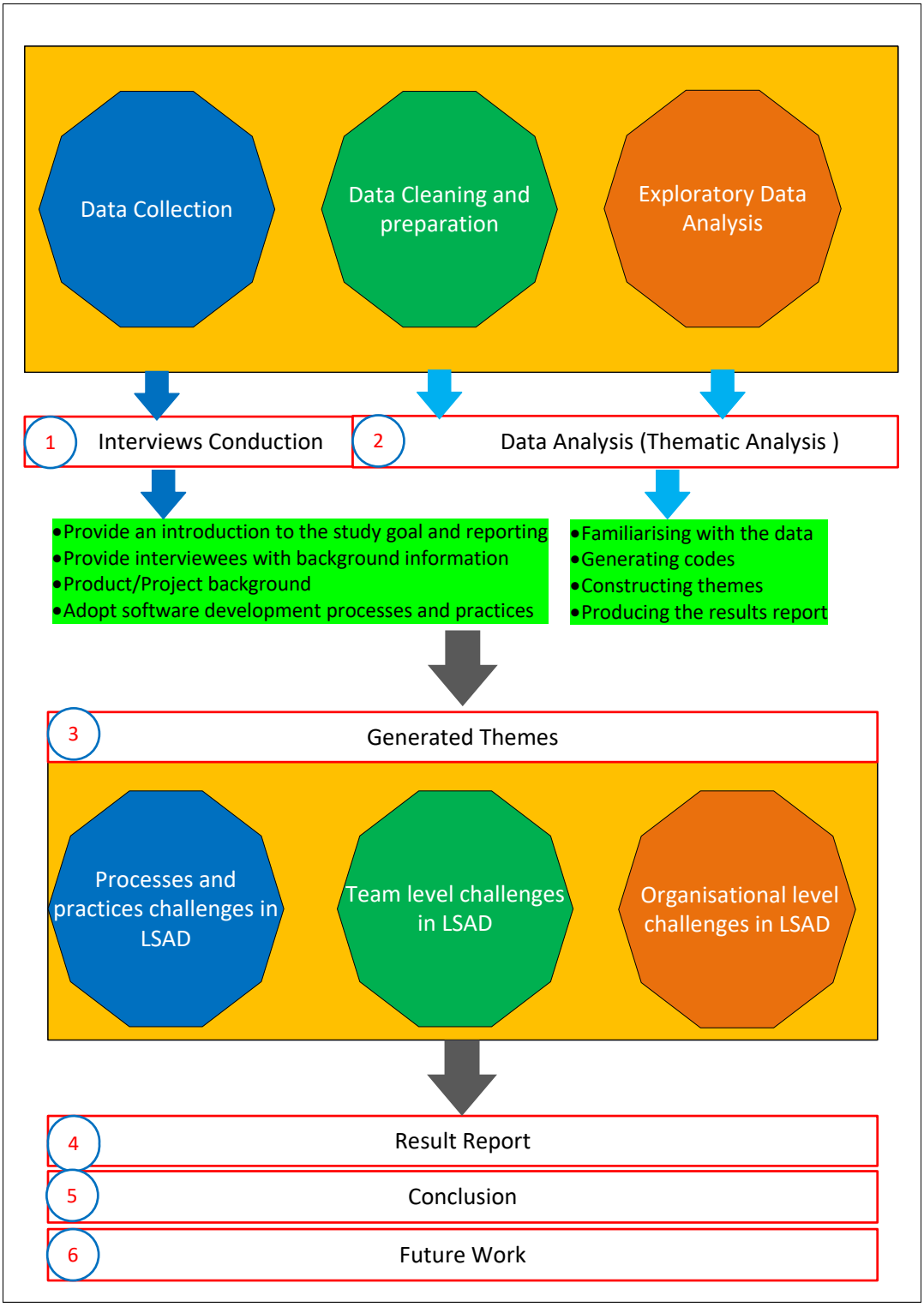

Figure 1: Research Method Followed.

Table 2: Agile elements used in case companies (Yes=✓; No=×)

#	Agile (Elements)	J	H	Remarks
1	Two-week sprints	✓	✓	CaseH has two to three weeks of sprints variations in different teams
2	Daily stand-up meetings	✓	✓	CaseH has daily stand-up meetings (only three times a week)
3	Planning meetings	✓	✓	None
4	Maintenance meetings	✓	×	Maintenance meetings only in CaseJ (SCRUM master and stakeholders involved)
5	Rotating SCRUM master role	✓	✓	The dedicated SCRUM master role is missing in the majority of case H teams
6	Product owner role	×	✓	Missing product owner role in some teams in CaseH, while completely missing in CaseJ
7	Prioritized product backlog	✓	✓	None
8	Retrospectives	✓	×	Multiple sprints with missing retrospectives in CaseH because of time and tight deadlines
9	Refinement meetings	×	✓	Missing grooming meetings in CaseJ due to missing product owner role
10	Requirements prioritization	✓	✓	Based on the highest priority, several tasks were selected for each sprint, but no formal prioritization techniques were followed in both Cases

taken into the next sprint). In CaseH, teams also face difficulties with retrospectives due to time constraints.

We identified 26 challenges categorized into three main themes (i.e., challenges in LSAD processes and practices, challenges in LSAD teams and organizational-level challenges) in LSAD.

3.4 Process and Practices Challenges

Table 3 summarizes the details of identified processes and practices challenges along with their descriptions. All of these identified challenges are divided into five major themes. i) agile testing challenges, ii) sprint challenges, iii) quality assurance challenges, iv) deadlines challenges, and v) requirements elicitation challenges. The themes mentioned are unique and newly discovered in the context of the case companies studied.

3.4.1 Testing Challenges

In the interviews, participants (PJ1- PJ4, PH7, PH8, and PH10) pointed out several frequent challenges associated with testing. Explanations for these challenges are provided below, along with examples drawn from case companies.

Lack of System Testing: In system testing, the behavior of the whole system/product is tested as defined by the scope of the development project or product. Because of a complex system with many components, testing of the entire application is missing, as testers ignore some aspects of system testing (i.e., ignored front-end testing). This happens as the teams lack a holistic view of the entire system. The interviewee expressed a lack of system testing as follows: *"Unfortunately, there are test suites, and there are loggers and error-handling libraries out there, but for some reason, front-end development, in general, shuns away from it. So it's part of this planning package for the lift, but not at the moment. We have maybe four [automated] test [cases], and they're very dumb"*(PH10).

Also, the issue is that testing the whole application is not completed before moving on to new tasks, as people forget to finish previous tests. With multiple teams working on different components of the system, it becomes even more complicated as they lack a holistic view of the entire system. Another interviewee expressed a lack of system testing as follows: *"We had faced this issue of being not completely done with testing the entire application things before we start new stuff. Everyone is interested, keen, or just curious to start a new test case forgetting to complete the previous one. Further, it gets much more complicated with several teams to test the whole application because different teams work on different system components without knowing the holistic view of the entire system"*(PH8).

Lack of Acceptance Testing: The process of confirming that a solution works for the user is known as acceptance testing. This helps to verify that the software does not crash and meets the specified requirements (i.e., tests that the user accepts the solution). It is the last phase of the software testing process, where actual users test the software to make sure that it can handle required tasks in real-world scenarios. However, in large projects, the problem arises when the actual end user is too busy or unavailable to perform the acceptance testing. The case organization is experiencing a

Table 3: Processes and practices challenges in LSAD

Challenges	Description	Participants
Lack of Testing		
Lack of system testing	In system testing, the behavior of the whole system/product is tested as defined by the development project or product scope. Because of a complex system with many components, testing the entire application is missing, especially on the front-end side.	PJ1, PJ2, PJ3, PJ4, PH8, PH10
Lack of acceptance testing	Acceptance testing is the last phase of the software testing process, where users test the software to ensure it can handle required tasks in real-world scenarios. The clients cannot do the acceptance testing because of a lack of resources. They are also not approving timely the developed features.	PJ2, PH10
Lack of regression testing	Different teams build system components separately. They lack regression testing and usually ignore that a change or addition made to the components has not broken any existing functionality of the system.	PH7, PH8
Lack of integration testing	Creating the different components of a large software system is the responsibility of different development teams. These teams cannot test system units, modules, and components of the system as a whole. As a result, they are unsure how these elements will affect and interact with each other in the future. As a result, different system components failed to work collectively.	PJ3, PJ5
Sprints Challenges		
Un-prioritized sprints	As all of the work in the backlog cannot have the same priorities, we need the order to do the most important work first. The product manager, customers, and key stakeholders set work priorities. Their lack of interest or un-involvement can lead to un-prioritized backlogs	PJ3, PJ5, PH6, PH10, PH11
Poor documentation	The lack of documentation in software development projects can pose significant challenges for teams and stakeholders involved. Documentation plays a crucial role in providing a clear understanding of the system functionality, processes followed, and the integration of different components. Without proper documentation, several issues can arise.	PJ3, PH6, PH10
Blind sprints	Accurate estimation of sprints is crucial in agile software development as it enables teams to plan and allocate resources effectively, set realistic expectations, and ensure timely delivery of the product. Sprints are not estimated well, leading to ambiguities in teams about whether a product is developed and delivered on time. For product owners specifically, poorly estimated sprints cause mismanagement of resources	PJ4, PH10
Pending sprints in the backlog	The product owner pushes the development team to start new cases while ignoring the old ones already in the backlog, leading to pending cases. This usually happens because the team is over-occupied. As the team consists of only two members.As the product owner focuses on new cases, older cases in the backlog may be overlooked or given lower priority. This can lead to frustration among team members who are responsible for delivering on these tasks.	PJ4, PH10
Quality Assurance Challenges		
Ignored coding standards	If coding standards are not followed, the resulting code is inconsistent and cannot be easily maintained. Smaller teams under time pressure skip the coding standards to rush the deployment	PH7, PH10
Unstructured code	Unstructured code refers to code that lacks proper organization and modularity. It is characterized by a lack of clear separation of concerns and the absence of well-defined modules or components. Unstructured code tends to be tangled and difficult to understand, maintain, and extend.	PJ1, PH11
Deadlines Challenges		
Strict deadlines (time limitations)	Strict deadlines in software development projects can have significant implications for teams. The pressure to meet these deadlines often leads to various challenges and compromises in the development process. One major consequence is the need for teams to strive for extra time to complete tasks, which can result in rushed development. Insufficient time for thorough development can lead to errors and defects in the software.	PH7, PH10
Missed deadlines	The discrepancy between the estimated effort of software development and the actual effort results in incorrect deadline setting. In addition, the unclear dependencies among different teams make it worse to predict the exact deliverable time for all teams.	PJ2, PJ5
Requirements Elicitation Challenges		
Changing requirements challenges	In software development projects, managing requirements is a critical aspect that significantly impacts the success of the project. However, one common challenge faced by teams is the continuous change of requirements without proper advanced communication. This dynamic nature of requirements poses several difficulties and can have a significant impact on the project scope.	PJ4, PH10

lack of acceptance testing due to the inability of customers to perform acceptance testing. The interviewee expressed a lack of acceptance testing as follows: *"The client's organization is affecting our work as we have to do acceptance testing on their behalf, and they have very few resources for this. So that's also a bottleneck that they don't have the time to approve what we are delivering"*(PJ2).

The acceptance testing process is complex and difficult to understand and perform. It is unclear when the activity starts and ends, making it difficult to know when to stop testing. End user unavailability is another challenge, requiring the testing to be done internally. The team develops and tests each other's functionality before moving to a separate test environment. Another interviewee expressed a lack of acceptance testing as follows: *"We are having problems with the acceptance testing. The tests are so complex to understand and perform... So it is hard to know when the activity started and was completed. Starting an acceptance test might be easy, but how it is completed and where to stop it is unclear. Another problem with acceptance testing is the unavailability of the end user; we need to do the acceptance testing on our own for the end users. We do it by developing the functionality, testing each other's functionality, and then moving to a different environment, called the test environment"*(PH10).

Lack of Regression Testing: Regression testing involves rerunning prior test cases to ensure the program continues performing as intended when updates or new functionality are introduced.

It is more complex and difficult in large projects with multiple teams to keep track of all changes. It can be a problem if the teams lack skills in testing and the company is unwilling to invest in testing procedures. The interviewee expressed a lack of regression testing challenges: *"For about ten years, we have been under-delivering quality-wise towards the client because much of the stuff that usually surrounds development has not been in place. For example, reasonable documentation and good regression testing of the application before releasing it is missing"*(PH7).

The case company's regression testing process is inadequate, with almost zero automotive tests performed. At the same time, manual regression testing is seen as a waste of time and prone to errors. The company needs to develop a solid regression testing suite, particularly for its major applications. Another interviewee expressed a lack of regression testing challenges as follows: *"Regression testing is not up to the mark in our company; we do almost zero regression testing. Any manual regression testing is a waste of time. But it's also even more error-prone if you must do it. So we need a solid regression testing suite for at least our major applications"*(PH8).

Lack of Integration Testing: The teams cannot test system units, modules, and components of the system as a whole. As a result, they are unsure how these elements will affect and interact with each other in the future. Lack of testing skills, strict deadlines, and teams under pressure can lead to a lack of integration testing. The interviewee expressed a lack of regression testing challenges:

"We only opened and ran the system six times, and it broke because the technology we used did not work in accordance with the cashier machine hardware platform. So we had to throw away all the code and start from scratch to develop the module. The reason for failure was doing integration tests too late to see the compatibility issues"(PJ3). The company lacks continuous integration and deployment testing, possibly due to teams not having a centralized way of working with testing and lacking knowledge of how to set it up. While some testing is done on individual components or products, there is a lack of integrated testing to ensure everything is working together. Performance testing is conducted at a high level, but integration testing is random, with no clear competence or time allocation for it. Another interviewee expressed a lack of integration testing challenges as follows: *"We do not do a lot of continuous integration and deployment testing. I think it has to do with teams not having a centralized way of working with the testing; they have less knowledge about how to work with the test and how to set it up. So they may have some testing in their component or product or deliverable, but they are not putting things together as a whole or checking how they are working when integrated. I think they have a very high level of performance testing, or at least a little bit, but they are very random regarding the coverage of integration testing. They don't have that competence, or even if someone has it, they don't have the time to put it into it"*(PJ5).

3.4.2 Sprints Challenges

In the interviews, participants (PJ1, PJ3 - PH6, PH10, and PH11) highlighted several issues frequently arising during sprints. Listed below are the identified challenges.

Un-priortised Product Backlog: Prioritizing the product backlog in the right order optimizes development time. The product manager, customers, and key stakeholders set work priorities. Their lack of interest, lack of knowledge, or lack of involvement can lead to un-prioritized backlogs. Teams not following formal prioritization techniques, such as story points, generate un-prioritized product backlogs. Stories prioritization in product backlog can be complex and difficult in large projects as it requires active communication and coordination among the product manager, customers, and key stakeholders. Also, backlog refinement and prioritization beforehand to identify and resolve dependencies among multiple teams is crucial to avoid team conflicts. The interviewee expressed un-prioritized product backlog challenges as follows: *"We're not using any story points for requirements prioritization. So, many things are new to people that we don't know how to estimate and prioritize in our product backlog since if you haven't done it before, how can you estimate and prioritize precisely?"*(PJ5).

The case company struggles with prioritizing cases that require time estimation and analysis. Currently, only one team member can analyze the cases because the other team members lack the required domain knowledge. Another interviewee expressed un-prioritized product backlog challenges as follows: *"What doesn't work that well is that we have many cases that need to be prioritized. And those cases need time estimation and analysis before prioritizing. Currently, only me is analyzing the case because the other team members don't have the domain knowledge to that level"*(PH6).

Poor Documentation: Agile usually has some necessary documentation to maintain the stability of software projects. However, it promotes collaboration rather than documentation as the preferred way to share knowledge. Problems arise when there is a lack of documentation about the clear breakdown of requirements, project milestones, the performance of the whole system, and the integration of its different components. The interviewees expressed the lack of documentation challenge as follows: *"We have very bad quality in the documentation about how the systems look, how it performs different functionality, and how the different parts of the systems in the form of the components are connected"*(PH6).

The team was given an Excel document with 300 lines of requirements with no clear breakdown or milestones. Despite being warned that this would cause issues, the people who ordered the work were okay with it. , But It is unclear how long the project will take or what the teams are working on. The lack of clear documentation is problematic, and the team needs to prioritize clear and concise documentation to ensure project success. Another interviewee expressed the lack of documentation challenge as follows: *"We had the Excel document with 300 lines of requirements with no proper breakdown, and the people who ordered this work were okay with it. 'Now we are done, that's it... and I told them initially that we cannot do it like this. The document needs to break down into clear requirements and milestones. I tried to tell them I could not do it just by looking at 300 uncategorized lines. I don't know how long we're at and what the teams are working on. We're going to see that at the end of this project going to miss much stuff here. We should not work like this ... we need to do clear, concise documentation to follow"*(PJ3).

Blind Sprints: The lack of sprint estimation confuses the team about how much work to bring into the sprint. It leads to failure in splitting product backlog items into small, discrete tasks. Consequently, the team is not able to assess the workload properly. The main reasons for the lack of sprint estimation are the lack of formal procedures to follow and the excessive number of haphazard changes in the product backlogs. The interviewee expressed the blind sprint challenge as follows: *"When we are starting a new sprint, and we have lots of cases that don't have a time and effort estimation, it's really hard to start the new sprint. It's a blind sprint. We don't know how big it is, how much time it will need, or how much effort is required actually"*(PH10).

The team does not have a proper estimation method in place for their work. Sprint estimations are done using story points, but very little progress is made in the first week, and a lot of changes are made in the second week. As a result, the team feels that doing sprint estimations is a waste of time. Another interviewee expressed the blind sprint challenge as follows: *"We do not follow proper estimation methods in our work. When we plan and do an estimation for a sprint and have all of these story points we can follow, almost nothing comes from the site in one week. And the next week, we have a million tons of things coming in, changing everything. It felt like it was a waste of time to do the sprint estimation every time"*(PJ4).

Pending Cases in the Product Backlogs: The cases can go into the pending state if the developers are continuously changing their working context (i.e., jumping from one task to another frequently). The other reasons for pending cases in the product backlog are associated with late information sharing among interdependent teams about the tasks. Further, the problem with the sprints happens when the product owners impose new cases to be completed before the old ones are already in the backlog. The interviewees expressed the pending cases challenge as follows: *"We are only a few people working in a team. So it's always a struggle when we have a new sprint to tell the product owner we can't manage more. This case has been in the sprint [backlog] for two sprints, and we haven't started yet. You can't put anything new in"*(PH10).

In teams, developers often start work on tasks within a case, but sometimes they require input from others. If the necessary input does not come on time, the developers cannot proceed with the task, and the case is put into pending status, causing the task to be put on hold. Another interviewee expressed the pending cases challenge as follows: *"Sometimes the developers start work on tasks in a case, but that task needs more input from others. So if that input does not come on time, the information doesn't reach the developers to proceed with the tasks. In return, the case went into the pending status, and this particular task in the development process went on hold"*(PJ4).

3.4.3 Quality Assurance Challenges

The interviewees (PJ1, PH7, PH9, PJ3, and PH10) mentioned several challenges that frequently arise in quality assurance.

Ignored Coding Standards: Coding standards, if skipped, create more complexity, increasing error chances. If coding standards are not followed, the resulting code is inconsistent and can not be easily maintained. Smaller teams under time pressure skip the coding standards to rush the deployment. The interviewee expressed the ignored coding standards as follows: *"So you could say that the coding standards that were supposed to be followed earlier were neglected by the people wanting new things to be done. So you made it too fast and did not with enough quality checks. After getting errors, now you're trying to get back and follow the coding standards and make a better product out of it attaining the quality"*(PH7). One of the team has been unable to follow proper code standards and conduct code reviews to ensure a certain level of quality. As a result, they often rush through development and end up with a product that "kind of works," which they then abandon. Another interviewee expressed the ignored coding standards as follows: *"So, following code standards and doing proper reviews of each other's code to ensure that we meet some quality standard hasn't been possible. This is doing our development in a rush, we end up with something that works, and we throw it out and forget about it"*(PH10).

Unstructured Code: The poorly written and structured code lacks modularization, making understanding, reusability, and testing difficult. The interviewee expressed the unstructured code challenge as follows: *"I think we have a pretty good pipeline for release handling. It can be better if we do code modularisation... But, to make things easy because it's a pretty complex system. So I would like to modularise the*

code and make it more structured and accessible for better release handling and testing" (PJ1). Another interviewee expressed the unstructured code challenge as follows: The teams have observed differences in their application's code structure, both in the front-end and back-end, which creates difficulties when trying to combine the code. These differences are due to different developers' styles and structures, resulting in team inconsistency. *"We have seen many differences in our applications codes inside out, such as differences in back ends and front-ends codes structures, which created a problem in combining codes at the end. These codes are created by different front-end and back-end developers. Everyone in different teams has their structure and tries to follow their styles, so differences arise"*(PH11).

3.4.4 Deadlines Challenges

Interviewees (PH7, PJ5, and PH10) highlighted several different deadline challenges, such as:

Strict Deadlines Challenges (time limitations): Strict, demanding deadlines leave little room for managing unforeseen problems. It can cause unhealthy stress, erode clients' trust, constrain creativity, and decrease team morale. These types of problems can burden teams, preventing them from being creative. Software developers say they feel pressure to sacrifice code quality to meet deadlines. The interviewee expressed the issue of the strict deadline as follows:

"It is not that the teams are not trying hard to meet the deadlines, but they can be behind schedule because of the strict deadlines. It is like a few people doing more tasks in a short time. Strict deadlines are problems; teams usually ask for more time to complete the tasks"(PH7). Teams feel that the deadlines set by the client are too strict and do not allow for creativity or trying new things. The team constantly works to meet these deadlines and does not have time to think outside the box. There is also a backlog of tasks that is never cleared, adding to the pressure. Another interviewee expressed the issue of the strict deadline as follows: *"I think that the deadliness that we're getting at the moment, or the expected timeline of deliveries from the client, is a little bit too steep and strict. We have a board that never gets cleared, and we always work against that. So I would say we don't have that time to think out of the box, try new things, or add some creativity"*(PH10).

Missed Deadlines Challenges: The discrepancy between the estimated software development effort and the actual effort results in incorrect deadline setting. Further, it is difficult to predict the exact deliverable time for different teams due to unclear dependencies. The interviewee expressed the missed deadlines issue as follows: *"We depend on other teams to deliver on time. We have a hard time meeting deadlines because It's a challenge to get the teams to deliver in time and then wrap it all up and pack agents ourselves before we can provide it to the client on time. So we're lost in the chain of dependencies and meeting the deadlines"*(PJ5). The team overestimates the number of tasks they can complete in a sprint, resulting in unrealistic deadlines that they fail to meet. Another interviewee expressed the missed deadlines issue as follows: *"We overestimate the number of tasks we do in a sprint. We set deadlines for sprints and continuously add too much stuff in the iteration of the sprints. That's making meeting the set deadlines unrealistic. Then we do not manage to deliver what is said"*(PJ2).

3.4.5 Requirement Elicitation Challenges

Agile projects on a small scale welcome continuous changes. However, in large projects changing project requirements are a common occurrence that might jeopardize project performance. Clients do not always know what they want and may develop better ideas afterward. They frequently do not understand how their changes in requirements will affect the overall project scope or what it takes to implement them. Moreover, in large-scale projects where multiple teams are involved tackling changing requirements is not easy. As in multiple teams, if requirements changes are not communicated and discussed timely, they can lead to team conflicts. The interviewee expressed the continuous changes and challenges as follows: *"So one of the big problems in our project right now is adding features and changes in the requirements. You have your project scope well defined, and then you keep filling it up with new requirements and continuous changes. That is causing the project scope to change and be everlastingly extending, expanding, and we can't ever find our way to the bottom to close"*(PJ4). The client frequently requests additional features, and priorities change abruptly, causing disruptions in the development process. Despite attempts to improve the requirements elicitation process, sudden changes still disturb the teams. The interviewee expressed the continuous changes and challenges as follows: *"The client adds more features and says, oh, there's this feature that we would like to have early. The teams escalate it so fast, as this comes from the very top. We need to implement this immediately. Unfortunately, we've seen that a lot because of these sudden unplanned changes in the requirements. We reiterated our whole requirements elicitation process many times"*(PH10).

3.5 Software Development Teams Challenges

Table 4 summarizes the details of identified Software Development Teams' challenges and their descriptions. These identified challenges are grouped into 4 themes team structure and composition, interdependent teams, lack of knowledge management and sharing in teams, and communication, collaboration, and teamwork.

3.5.1 Team Structure and Composition

Interviewees (PH3, PH7, PJ2, PJ3, PJ5, and PH10) highlighted several different team structure and composition challenges, such as:

Continuously Changing Teams: Continuous changes in Agile teams arising from different factors can raise many challenges. The frequency of these changes can impede the flow of work and adversely impact project timelines. Secondly, the dynamic nature of team membership renders it difficult to ascribe clear responsibilities, leading to confusion and reduced accountability. Additionally, the departure of experienced personnel results in a loss of knowledge and skills that may not be easily replaced, necessitating further training for incoming hires. The interviewees expressed the continuously changing team's challenge as follows:

Table 4: Team level challenges in LSAD

Challenges	Description	Participants
Team Structure and Composition		
Continuously Changing teams	Continuous team changes can significantly impact project timelines, team morale, and cohesion. The dynamic nature of team members introduces confusion and reduces accountability, ultimately leading to decreased productivity and increased stress levels. Maintaining a stable team structure and minimizing frequent changes is crucial for ensuring smoother project execution and fostering a positive work environment.	PH7, PJ3
Managing Workload and Expertise	Managing workload with fewer expert and skilled team members available for specific tasks results in difficulties handling the entire infrastructure. This causes uneven workload distributions. Lack of understanding about the system among the team members also hinders successful agile software development. Thus, balancing the number of expert and experienced team members is crucial.	PH7, PH10
Distributed Teams	Remote work poses challenges when team members are located in different locations. Communication can become more difficult, and team members may feel isolated if they lack regular interaction. It can also be challenging to stay informed about the work status and progress of team members. Finding effective ways to foster communication, collaboration, and transparency is crucial to overcome these challenges and maintaining productivity and engagement within remote teams.	PH6, PH10
Interdependent Teams	Working in a complex and extensive system with interdependent teams presents challenges in comprehending the system as a whole, identifying the responsible team for specific applications, and resolving problems in a timely manner. These difficulties can lead to delays in project execution, hinder effective collaboration, and impede the overall progress of the development process. Finding effective ways to streamline communication and clarify team responsibilities is essential for overcoming these challenges and ensuring smoother project coordination.	PH9, PJ2
Delayed Tasks and Team Conflicts	Interdependencies among teams can lead to delays in task completion and team conflicts. When one team relies on others to deliver on time, meeting deadlines becomes challenging. To enhance success, speed, and agility, it is crucial to have sufficient developers within the team who can handle their tasks independently, reducing dependence on other teams to implement specifications and ensuring smoother workflow and timely delivery.	PJ2, PJ3
Lack of Knowledge and Sharing in Teams		
Lack of domain knowledge	The lack of experience and domain knowledge in new teams hinders their ability to collaborate effectively, despite their enthusiasm. Their focus on individual tasks often creates difficulties when trying to work together as a cohesive team. It is crucial for team members to acquire sound domain knowledge to enhance their collective abilities and foster collaboration. By developing a shared understanding of the project, the team can overcome challenges and achieve greater synergy in their work. Cultivating a collaborative environment that encourages knowledge sharing will enable the team to leverage each member's expertise and maximize their collective potential.	PJ3, PJ4
Lack of knowledge sharing	Insufficient knowledge sharing among team members poses a significant challenge in software development. The lack of communication and collaboration hampers their understanding of the project. The absence of formal knowledge-sharing meetings further exacerbates this issue. Additionally, team members often fail to share their knowledge with other teams, despite having interdependent tasks. This lack of knowledge exchange leads to decreased efficiency, potential misunderstandings, and missed opportunities for improvement. Addressing this challenge requires fostering a culture of open communication and collaboration within and between teams.	PJ3, PH6
Lack of Communication and Collaboration		
Communication	The team's communication was ineffective. The team tried experimenting with ways to improve communication, especially because they worked in different locations. They preferred having discussions on calls rather than writing in the chat. The same team also mentioned a challenge in communicating with another team, "T," who worked on the booking system. They experienced difficulties because the other team did not inform them about changes they made before going live, causing issues with the data.	PH6, PH7
Collaboration	The team is having trouble collaborating with other teams. One reason is that they have many dependencies on another team, meaning they rely on that team to complete tasks before they can do their own work. However, the collaboration is not working well, mainly through their product owner. Additionally, they don't communicate much with other teams or know how to improve the use of other teams' skills.	PJ2, PJ3

"And this is something [continuously changing teams] we have been paying attention to in the last few years. There are many reasons for cautiously changing teams. There were basically, when the pandemic struck; we lost a lot of people. There was an earthquake in the working market, and we lost and gained many people. We hired new people, but you can't replace heads, but you can replace experience in a certain project. These continuous changes in teams can lead to uncertainty and instability. When we lose old employees, we also lose many competencies, and by new hiring, we need to provide additional training. With continuously changing teams, it's hard to track the responsibilities; who is responsible for what?"(PH7).

Role changes can be difficult for team members, and it is important to be respectful and not push things through just because they might be better. It is important to ensure team members understand why changes are necessary and willing to try them. If they don't understand or don't want to, the changes will not work.

Another interviewee expressed the continuously changing team's challenge as follows: *"At that point, I'm like, of course, this is a significant change for everyone involved if someone is leaving in between or there are changes in the roles. I have faced this many times in my team. I know that everyone takes their time to get some changes in their roles, and some people never want to do it. So you have to be respectful and not try to push things through just because you know it can be better. That's the hard part of forcing team members to some changes. They need to understand why and if they now understand why and are willing to try it, it will work. But it will not work if they don't know why some people don't want it; it will not work. So it's from within the team"*(PJ3).

Managing Workload and Expertise in Agile Teams: Managing a workload with fewer team members available for specific tasks or fewer skilled or expert team members for specific tasks causes many project challenges. These challenges can lead to problems in team members struggling to know enough about the entire system. Having a team with fewer expert members can result in overworked team members. This can also cause putting important responsibilities on the shoulder of a few of the team members. Finding the right balance between team size and expertise is essential to address these challenges and ensure successful agile software development. Successful agile software development. The interviewees expressed the managing workload and expertise in agile teams challenge as follows: *"These few persons in teams have been trying hard the last few years to lift the entire infrastructure and how we do things. It is a course of many problems related to the fewer people available for specific tasks in teams than we have today in the project challenges. These fewer persons also struggle to manage the workload in a team. So for a couple of years, our main challenge has been having enough expert team members with knowledge about the entire system"*(PH7).

Also, some teams were previously backend-heavy and struggled to balance backend and front-end tasks. After a new member with front-end skills joined the team, they could move faster on front-end tasks. However, they still lack expertise in some areas and may sometimes need to consult with someone with a better technical understanding.

Another interviewee expressed the managing workload and expertise in agile teams challenge as follows: *"Earlier, we were only a backend-heavy team. We struggled to manage technical expertise as we were back end heavy team. We still used to handle the front-end cases along with the backend, which was hard for us. As we have a heavy backend and a lot of backend skills, we tend to take back-end cases first and give less attention to the front-end tasks. And after this guy joined, the front end also moved quickly. But sometimes, we lack system understanding because of a lack of expertise in our team. Sometimes we end up in that situation at the time. We are just in touch with someone who better understands technical consulting and technical aspects where we want experience competence"*(PH8).

Distributed Teams: Agile team members face several challenges while working from distributed locations. Specifically, team members being located in different remote locations can make it difficult to catch each other on time and can lead to a feeling of isolation. If communication is not maintained regularly, it can be unclear whether team members are working or experiencing other difficulties. The interviewees expressed the distributed teams' challenge as follows: *"I think, but because our team members are located in locations X and Y. So, everyone is in remote locations and doing remotely. So that can be a difficulty in catching each other on time. Sometimes if we don't hear from each other on MS Teams, it may feel a bit isolated, and we don't know if this person is working or maybe not feeling well or stuck. That can be an issue. Sometimes you can bail out. I try to be daily on MS Teams, write to the person and ask: "Is everything going fine? Are you blocked"*(PH6). The team works remotely, with some members located in different physical locations. They believe working in a co-located setting could make collaboration and communication easier. Also, there can be informal information exchange between team members who are physically located together, which distributed team members might miss out on. Another interviewee expressed the distributed teams' challenge as follows: *"We're remote. Some other guys are sitting together in location X, but I'm down in location Y, so it's all between teams. But I know if I'd been, there probably would have been easier to sit down next to someone to ping someone; working from co-located places could have made it easier to have some steps sort of be introduced to other team members. So working remotely is about, that you are sitting in the corner. So it's mainly a distributed environment in our team. Maybe it's a little bit. If exactly everyone is distributed in other projects, it's easiest. But then some also sit together, and there's also some informal information going on there that you miss out on"*(PH10).

Interdependent Teams: Working in a large, complex system with multiple applications and interdependent teams can pose many challenges. The new employee entering the system can take more time and effort to understand the system. The team members can not get enough opportunities to communicate with everyone on the team or with codependent teams. Identifying the appropriate team responsible for a particular application can be difficult, leading to confusion and misunderstandings. Determining the responsibility for

resolving problems and whom to contact for a solution can also be challenging, resulting in delays.

The interviewees expressed the Interdependent Teams in Complex Systems challenge as follows: *"I am new in this team. It's an extensive system with many applications and bigger and codependent teams working on it. So it could just be that I haven't gotten a clear grasp of it yet[the entire system] and did not get a chance to talk to everyone[the entire team]. But sometimes, when problems occur[in large extensive systems], it could be a little hard to see where the responsibility lies[what teams are responsible for it and to whom to contact for the solution]. This can result In misunderstandings and delays"*(PH9). Another interviewee expressed the Interdependent Teams challenge as follows: In interdependent teams, It's hard to see where other teams are with things working in co-dependencies. They have sync meetings, but the team members don't hear or see them often. Using different issue management systems is another problem for them as a dependent team to follow the common tracks. *"We have many dependencies, especially on one other team. We're often the base for everything else, the data or the database layer. So many people want stuff from us. So to speak, the communication there, it's, well, it's mainly through our product owner in the team, and it's not very formalized. But that's a bit of it's hard to see where other teams are at and with things involving us. There are sync meetings that the RPO is involved in weekly or bi-weekly or something like that. But it's not something that I hear or see often. There is much to see what's going on at a higher level; another problem for us as a dependent team is using different issue management systems"*(PJ2).

Delayed Tasks and Team Conflicts: In large-scale software development projects, multiple teams are pushed to work collaboratively on various inter-depend tasks. The dependencies between teams can be task-based and outcome-based. In large-scale agile projects, it is difficult to locate task orders, understand why specific tasks depend on each other, and create a document that maps all dependencies, providing a clear overview of the project. Some significant reasons for team interdependencies are lacking resources, tight deadlines, and large and complex projects with multiple modules. If not appropriately managed, the interdependencies between teams can delay tasks, cause conflicts, and negatively affect productivity. An interviewee explains this challenge of delayed Tasks and Team Conflicts because of strong dependencies among teams as follows: *"So our team has many dependencies on other teams, and we depend on other teams to deliver on time. We have a hard time meeting deadlines because It's a challenge to get the teams to deliver in time and then wrap it all up and pack agents ourselves before we can provide it to the client on time. So we're lost in the chain of dependencies and meeting the deadlines. Because of this, we face conflicts with other teams"*(PJ2). Another interviewee explains this challenge: *"So for my team to be more successful, we need a couple more developers who can do our team's stuff. Because right now, we specify things for other groups to implement. If we want more speed and agility, we need to cut the dependencies. We need to be self-dependent. We need to make our stuff; we cannot do that now. We cannot do everything. We can do some stuff, but maybe 50 percent land outside us. So that's a fair amount of dependencies, and we are supposed to be the ones who drive things to be done and commit to this. And we need other teams to achieve savings and have their stuff. It's hard and affects our timelines"*(PJ3).

3.5.2 Lack of Knowledge Sharing in Teams

Interviewees (PJ3, PJ4, and PH6) highlighted several different lacks of knowledge management and sharing challenges in teams, such as:

Lack of Domain Knowledge: Software development requires explicit domain and episodic or tacit knowledge (experience-related knowledge). To complete a software development task, team members must have a certain degree of both knowledge streams. Difficulties may arise when teams have an imbalance in the mentioned knowledge streams. As agile principles emphasize people over processes, individuals' knowledge is critical to project success in agile software development. However, problems arise when knowledgeable individuals leave teams, when a small group retains all knowledge, or when teams lack cross-domain knowledge. An interviewee expressed a lack of knowledge caused by the leaving of experienced team members as follows: *"... product owner created these user stories. But then we took them directly to refinement, and there were too many misunderstandings and challenges because we, as developers, thought, Oh, this is easy; we will do it. It's just adding a text field here for the first name. But then the product owner or the stakeholders that previously brought these requirements didn't have the full knowledge and experience of what needed to be done down the chain, so the detail was missing on how to implement them. As initially, they know how they want the customer to experience things. But when they start to dig into the details, for example, there were a lot of new requirements they need to know more about and understand practically to implement them"*(PJ4). The lack of domain knowledge negatively contributed to the cooperation with other teams. A manager expressed a lack of domain knowledge in the new team as follows: *"We're a completely new team and haven't done much work or do not have more domain knowledge, but it's exciting because they [new team] are very focused on their little world and not the whole world. So it's hard to cooperate with them as a team because, as a team, you want to utilize everyone you want to use everyone as a team"*(PJ3). Further, lack of domain knowledge becomes more complex when a team member stays in the company for too long and is now pushed to update his/her knowledge. As expressed by an interviewee: *"Everyone is a resource in the group to do the team's tasks. But, it's challenging when someone has 15 years of experience in software and no expertise or knowledge whatsoever in any other software domain. So, to utilize that kind of new resource with limited domain knowledge, you either need to educate them to have the basic knowledge, and they need to be open to being educated, or you need to have all these things bound to this system"*(PJ3).

Lack of Knowledge Sharing: In today's software industry, knowledge sharing becomes more challenging as diverse and multiple teams are involved in developing large and complex software systems. In the context of agile software development, Agile principles emphasize tacit knowledge only. Further, this issue is enlarged because of inadequate resources

and documentation to manage knowledge sharing in agile at scale.

Lack of Intra-Team Knowledge Sharing: Consequently, due to a lack of knowledge-sharing strategies in large-scale agile software development, the teams cannot see the holistic view of the system, making it hard to communicate technical dependencies between them. An interviewee expressed a lack of intra-team knowledge sharing as follows: *"In our team, knowledge sharing or learning from each other is not in any formal capacity. This is one thing that needs to be improved for a better understanding of the whole project. We do not have any meetings to exchange our knowledge with each other"*(PJ3).

Lack of Inter-Team Knowledge Sharing: Further, another interviewee expressed a lack of inter-team knowledge sharing as follows: *"We do not have any knowledge sharing things at all; I guess we are not perfect in it. We have no active knowledge sharing with other teams, although we have interdependent tasks with them. We have a shot with where all the teams are; some people sometimes write hey, mostly like hey, and this package has been hacked. We need to remove that from our interdependent tasks. And that's competence sharing at all, nothing more"*(PH6).

3.5.3 Communication and Collaboration Challenges

Interviewees (PJ2, PJ3, PH6, and PH7) highlighted several different communication, collaboration, and teamwork Challenges, such as:

Communication Challenges: Effective communication is essential for software development teams to facilitate knowledge sharing among team members. Frequent and efficient communication allows teams to understand customers' needs better. The agile principle promotes frequent face-to-face communication inside and outside the teams. However, frequent face-to-face communication is difficult in scale agile teams due to distributed nature of diverse teams, time zones, and working culture differences. In our case, large-scale software development companies lack effective communication mechanisms inside and outside the teams. This concerned both remote and onsite teams (inter-teams) and onsite teams (intra-teams). The communication is mainly managed via Microsoft Teams and text chat.

Intra-Team Communication Challenges: An interviewee expressed intra-team communication challenges as follows:*"We spend a lot of our days in MS -Teams and so on, but we do not have any rules for communicating within our team. Communication within teams is an issue. We talked about it, and we are trying to experiment a little bit, and we are. Since we are often at different locations, we have said that we want to remove as many barriers as possible to have more effective communication. So I have told my teammates that I don't want them to write in the chat, instead to discuss the calls to make communication effective"*(PH7).

Inter-Teams Communication Challenges: The interviewees also shed light on the inter-team communication challenge in their work as follows: *"We mostly need active communication with a team called (T). They are working on the booking system. The team we have the most contact with. That has happened recently, and they have upgraded their system and live everything. We have received lots of bad data from them that we were fixing on our side and everything. They have tested their things much better and completed their tasks but did not inform us. They should have informed us that they had made this change before they went live. That was a lack of communication and a bad thing on their side"* (PH6).

Collaboration in Teams: Software development is a complex team effort that requires efficient communication and effective collaboration. Agile promotes close collaboration with team members, but it becomes challenging when multiple teams interact on large-scale projects. In such a large-scale project, the risk is that most communication and collaboration become inward team focus and less concerned with connected teams. Interviewees express collaboration challenges as follows: *"Regarding collaboration, it does not work well as we have many dependencies, especially on another team. We're often the base for everything else, the data or the database layer. So many people want stuff from us. So to speak, the collaboration there it's not well. It's mainly through our product owner in the team, and it's not very formalized"*(PJ2). *"I would say we don't collaborate that much with other teams. We don't write a lot to other team members to need help. This grandmaster decided that we needed to improve using other teams' competencies. We don't know how"* (PJ3). It is important that companies develop strategies that promote effective collaboration inside and outside teams. Additionally, communication and collaboration tools used in large-scale projects require more attention. It must be uniform to sync the work and handle team task dependencies. Lack of collaboration inside and outside the team causes distress and worsens handling task dependencies. Another interviewee explains this challenge of collaboration with an example as follows: *"One problem is that we're using different issue management systems. So, we're not all using the same. I think mainly inland; the other teams use JIRA and Azure DevOps. So that's a problem; it's not easy to go to the related item in JIRA; maybe you do it. Up until recently, we didn't even have access to it. For some of the projects, I don't think that was prohibited. It's just that no one thought we should have that with us for collaboration. We finally got it when we asked for it. But it's hard to see where other teams are and with things involving us. There are sync meetings weekly or bi-weekly or something like that. But it's not something I hear or sees and know as much of it is going on a higher level"*(PJ2). *".... [we] start writing to people, and there I can see a problem with it being on MS teams and not in real life because it's easier to ask for help if they're in the office beside you, so collaboration with distributed teams or with remote work is a hurdle"*(PJ3).

3.6 Organisational level challenges in LSAD

Table 5 summarizes the software development organizations' challenges and descriptions mentioned by the Interviewees (PJ2, PJ3, PJ4, PJ5, and PH6). These identified challenges are hard getting feedback from client organizations, organizational fragmentation and lack of common structure, and delayed decision-making in the organization.

Hard to Get Feedback from Client: When busy, the client organization makes it difficult to get their feedback. While getting feedback is not impossible, it's still challenging to

Table 5: Organisational level challenges in LSAD

Challenges	Description	Participants
Hard to Get Feedback from Clients	The client organization is busy, and getting their feedback is challenging. The product owner is responsible for answering most questions on behalf of the client organization. But this feedback may not satisfy everyone involved in production. Lack of feedback during acceptance testing can lead to difficulty identifying and addressing user concerns, causing project delays and cost overruns.	PJ2, PJ5
Fragmentation and lack of Common Structure	The organization has separate setups in different countries following different working structures, causing difficulty in collaborating on projects. Different departments have a conflict between traditional and agile project management approaches, and the introduction of a new project requires collaboration. Still, the classic teams do not want to follow strict agile methods. This lack of common structure is causing conflict and is not ideal.	PJ3, PJ4
Balancing Technical Expertise with Managerial Responsibilities	In large organizations, people with technical expertise who become department managers can find it challenging to balance their managerial responsibilities. They may have developed many internal applications but cannot find time to work on them or delegate them to someone else. This can create problems for managers who cannot fulfill their technical duties, leading to team difficulties. Some people can find it very complex and hard to have both responsibilities. However, team members can ask for help and work together to find solutions, with the Product Owner (PO) assisting them by sharing their experience or taking on some of the work.	PJ4, PH6
Delayed Decisions	In large organizations, decision-making can be slow and time-consuming due to involvement from higher authorities. Product owners may avoid making decisions that require involving others to prevent delays and inefficiencies. Delays can also come from the client organization if they are indecisive about technology or lack clarity about requirements, leading to prolonged development.	PJ3, PH6

get answers to specific questions or to all of the questions. This is when the product owner is responsible for answering most of the questions on behalf of the client's organization. However, it's still hard to satisfy most people involved in production with this feedback. An interviewee expressed a lack of feedback challenges as follows: *"CaseJ is very busy, so getting feedback is hard. Now it's not hard to get feedback, but the questions, answers to questions, it's hard to get them. So I think that's why I have my role as a product owner. So yeah, I'm answering most of the questions, but it's hard to get a hold of people you know"*(PJ2).

Lack of feedback from the client organization during acceptance testing can be a problem because it can prevent the software or product from being improved or modified to meet the user's needs better. Without feedback, it may be difficult to identify and address any issues or concerns the user may have, leading to a product that the user may not fully accept or adopt. Additionally, a lack of feedback can cause delays in the testing process, leading to project delays or cost overruns.

An interviewee expressed a lack of feedback from the client organization on acceptance testing as follows:

"Well, their organization is affecting our work because we have acceptance testing on their part, and they have very few resources for that. Yeah. Extremely little help, I would say, So that's also a bottleneck that they don't have the time to approve What we're delivering. So that's also a bottleneck, actually, and a big one. -That I don't think they have realized yet. That I mean, it's not that they're halfway there when it comes to resources, they would need to triple or quadruple or much Stuff missing there, and that Is from my perspective coming from that side of it"(PJ5).

Fragmentation and lack of Common Structure: The case organization has different setups in different countries that work somewhat independently, making their own decisions and developing their own applications. In some areas, they work together and have common applications. However, when they try to work on a project together, such as updating an old application, there are differences in opinion and needs between the IT teams in different countries, making it difficult to get everyone on the same page and start the project. An interviewee expressed organizational fragmentation and lack of common structure challenges as follows: *"One of the biggest problems, I think, when it comes to the organization is that you have a CaseH globally, which we don't care about at all. We have CaseH Nordic, Sweden, Norway, Finland, and Denmark, somewhat in some areas. They are on their own and design, decide, and maybe even develop their own applications. And for some areas, they work together and have common applications. But when you have that situation, and you try, we have a good example that we have a very, very old application in our team, and it needs to be redone from scratch and this application. It's a Norwegian application for the Norwegian market, but the plan is to make it work for all Nordic countries. But as it stands now, I don't know if we can start this year because there are different opinions between the IT persons within the CaseH in different countries. So they and I guess every country has its own need and trying to get all this together"*(PJ3).

The case organization is facing a clash to follow a common structure of approaches to project management. Some departments and teams use the classic traditional approaches, and some follow the agile working method. Still, there is a push to introduce a new project that these different departments must collaborate on by following the agile way of

working. However, the teams using the classic methods do not want to be strictly agile and think the plans should be set in stone. This lack of not following one common structure is causing conflict between the teams and is seen as a bad situation.

An interviewee expressed organizational fragmentation and lack of common structure in different departments as follows: *"There are some different departments working together on a common product. They work on classic waterfall methods to follow for the projects. But some departments and teams working on the same common products follow agile approaches. So we are going to introduce a new product in between these departments. Then we need to take new packaging designs that start with product segment divides as Segment A, segment B, and Segment C. If we need to collaborate between all these segments, we need to start collaboration in the beginning before all departments and teams have planned all of their work. But some of them have started this two years ago. So here we have the classic sort of head clash about common structure following. We try to be agile, and the other departments and teams are the other way around. None of these projects or these departments understand an agile process. That's a bad situation"*(PJ4).

Balancing Technical Expertise with Managerial Responsibilities: In large organizations, people have problems balancing their technical expertise with managerial responsibilities. People who love programming and are developers but become department managers can find it difficult to like managerial responsibilities and still want to code. These people may have developed many internal applications but don't have time to work on them anymore, and they are also unwilling to hand over their started products to someone else. This creates problems for people with high managerial responsibilities as they keep promises to fulfill technical duties but can not do so. For instance, a manager still receives calls at night regarding the system in the warehouse because they cannot let go of their project.

An interviewee expressed balancing technical and managerial responsibilities challenges: *"I have two examples. One of the guys loves programming. He is a developer. But somehow, he ended up being a department manager for ten people and five people or something of developers. But he doesn't want. He developed a lot of applications, including internal applications. And those are not handed over to his staff. So when he has all this, he has to have all this manager responsibility, keeping discussions about people that feel bad or following up on things and planning vacations. You know, everything that the department mentions are at the same time. He thinks it's so funny and so. He likes programming so much that he wants to do this. But then the problem is he doesn't have time to do what he promised. And the other thing is, when we discuss changes in those applications, he doesn't have the time he needs. Because then he gets X picking on him if he doesn't do his payroll discussion. So we have, and that is a mentality within your whole organization. I have nursed this; I have created this. This is my baby. Letting go of your darlings in some way is missing throughout the organization. And there are a lot of people with high responsibility. We have the worst cases. We have a manager that still are on nightly calls. So when it system in the warehouse is, they call him at 3 a.m. because they are working 24 seven"*(PJ4).

Some of the team members struggle with having both technical and managerial responsibilities. They feel stuck in complex tasks and responsibilities. In that case, they ask for help. It is good for team members to feel comfortable speaking up when they have difficulties. The team can work together to find a solution, and the Product Owner (PO) may be able to help by sharing their experience or taking on some of the work. Another interviewee [PO] expressed balancing technical and managerial responsibilities challenges as follows: *"And they're actually, one member contacted me and said, "I'm not feeling very well now because of this thing of having both technical and managerial side responsibilities that I'm working on. I can't manage it. It's too complex for me ". So that's really good that no one is afraid to raise their voice when they don't manage the things they are working on. It is good to talk. If you are stuck and feel bad because of more responsibilities, maybe something is too complex. So we can sit together as a team, or maybe I can take part in it [as a PO] because I have the experience; maybe we can share the work"*(PH6).

Delayed Decisions: Decision-making can be lengthy and time-consuming in large organizations as it needs the involvement of higher authorities. Some decisions on the team level that only needs product owner approval can be fast. But once the decision-making responsibility is passed on to others, especially higher authorities, it becomes slow and delayed. So most of the time, the product owner avoids making decisions that require involving other people because it leads to more delays and inefficiencies in the decision-making process. An interviewee expressed delayed decision challenges as follows: *"I think so, yes that did not say that, of course, not all decisions are being dragged out some decisions are made very quickly, basically on the spot by the product owner. But when the decision leaves the product owner's domain, it instantly becomes very dragged out in time. So I can see when we talk to our product owner that as long as he feels that he can take that decision by himself and take it very quickly. Yes, but he dreads taking it out in the organization because it's like throwing it into a black hole. Yeah"*(PJ3).

Decisions can be delayed from the client organization side if they are not decisive in what kind of technology they need. Or they lack clarity about their requirements. This kind of decision delay can cause development to drag on for a long period of time. An interviewee expressed delayed decision challenge from the client organization side as follows:

"I can take an example of delayed decisions, but it's actually technical. We have developed an administration tool where the customer can do configuration and setting up customers and everything like that. And we should have the handling of accounts in that system, but we haven't had a decision from CaseH on what system we should use. So you need a decision: what technology should we use? But they haven't been decisive. It's development for us that has been dragging for ten months just because we haven't had a decision from CaseH. Because they can't decide what technology they should use"(PH6).

4. LIMITATIONS AND VALIDITY

We consider four aspects of validity threats: construct validity, internal validity, external validity, and reliability [27]. Construct validity is concerned with obtaining the right measures for the studied concept. One threat was the selection of interviewees to obtain the appropriate sample for answering the research questions. Therefore, experienced interviewees within the companies were selected, keeping the following aspects in mind: knowledge of agile software development and experience using agile at scale. For construct validity, interviewees were introduced to key concepts used in the study at the beginning of each interview session to avoid misinterpretation.

As this study was of an exploratory nature, internal validity was not considered. External validity is the ability to generalize the findings from a specific study context to other contexts. Data is collected from two companies to address the generalizability of the results. Inside these companies, different participants from teams with different roles are selected to cover the broader perspective. However, the sample size used in the study might not capture the full range of challenges encountered in large-scale agile software development projects across different industries and contexts. The study focuses on challenges encountered in large-scale agile software development projects in Swedish companies. The findings may not fully represent the challenges faced in other countries or cultural contexts, where organizational structures, practices, and project management approaches may vary. The findings should be considered with caution when applied to other contexts (i.e., small-size projects or organizations) where the agile way of working is followed. The threat to reliability describes generating the same result upon repeating the study with the same settings. There is always a risk that the study's outcome is affected by the researcher's interpretation. To mitigate this threat, the study was designed, and data were collected through interviews from two different case companies with different roles of the participants taken into consideration. Additionally, at least two researchers were involved in reducing the risk of bias in each phase of the research process.

5. DISCUSSION

The broad benefits of agile in small-scale projects, such as cost-effectiveness, early time to market, and dealing with constantly changing requirements, encouraged practitioners to use it in large-scale projects[6]. However, large-scale software development projects and bigger organizations with more work and team dependencies present various obstacles to the agile way of working[19]. The existing literature calls for investigation on agile scaling in industrial settings [11, 12, 15, 19, 21, 31, 30, 32, 33]. We conducted a case study to investigate the challenges encountered while completing large-scale agile software development projects in two Swedish companies. We performed eleven semi-structured interviews and analyzed them thematically. We identified 26 significant challenges categorized into three main themes (i.e., challenges in LSAD processes and practices, challenges in LSAD teams, and organizational-level challenges in LSAD). Most of the challenges identified are related to processes and practices (n=13) followed in LSAD, including testing complex systems in large-scale projects involving multiple teams. The case companies face significant challenges regarding testing strategies, specifically integration, regression, and user acceptance testing. Such test challenges lead to "test debt" [3]. Second, the teams encountered sprint-related challenges, which Ahmad and Tomas categorized as non-technical debt [3]. Non-technical debt is an umbrella term to cover social, process, and people's debts [3]. Process debt covers challenges such as lack of sprint prioritization, pending user stories in product backlogs, and blind sprint challenges, whereas inadequate documentation comes under the umbrella of documentation debt. Compromised quality assurance processes were reported as the thirdly highest process-related challenges, including complex and unstructured codes and ignored coding standards. Furthermore, missed deadlines, constantly changing requirements, and project scope are reported due to communication and coordination issues between various teams, product owners, and end users.

The current study further investigated the obstacles (n=9) encountered by LSAD teams in their software development projects. The findings revealed that continuous team changes and a lack of expertise could have a negative impact on project timelines, team morale, and cohesion. This is consistent with previous research indicating that team stability and expertise are essential for successful software development projects [25]. The LSAD team's continuous changes have resulted in confusion and reduced accountability, leading to an uneven workload distribution, decreased productivity, and heightened stress levels. The team's workload management has been challenging, with fewer expert and skilled team members available for specific tasks. This has resulted in difficulties handling the infrastructure, leading to decreased productivity and increased stress levels among team members. Moreover, working remotely with team members in different locations has posed communication and coordination challenges for the LSAD team. The team members may have difficulty communicating and feel isolated if they don't hear from each other regularly; knowing their work status and progress can also be hard. The complexity and interdependence of systems have further complicated matters for the teams. The LSAD teams have faced challenges in comprehending the entire system, identifying the right team responsible for a particular application, and determining the team responsible for resolving problems, resulting in delays.

The challenge of delayed tasks and team conflicts happens because of interdependencies among teams, where one team depends on others to deliver on time, leading to difficulties in meeting deadlines. It is essential to balance the number of experts and experienced team members to address these challenges and increase speed and agility. Ensuring better documentation, communication, and coordination among team members is also crucial for better documentation and knowledge sharing; automated extraction of business documentation in enterprise information systems can play a positive role. Our study suggests that increasing the number of developers who can handle tasks independently can also be helpful. Thus, building a stable and knowledgeable team ensures better communication and coordination and provides the necessary resources to increase efficiency

and productivity. Our study further highlights several challenges (n=4) that large organizations face following the agile working methods. One of the major challenges is the difficulty in obtaining timely and satisfactory feedback from the client organization, which can cause delays and cost overruns. The lack of a common structure and conflicting project management approaches in different departments can also result in conflicts and inefficiencies. Another significant issue highlighted is the challenge faced by technical experts who become department managers in balancing their managerial and technical responsibilities. This can lead to difficulties for managers who cannot fulfill their technical duties, resulting in team difficulties. However, this issue can be resolved by team members working together to find solutions with the assistance of the product owner. Furthermore, the involvement of higher authorities in decision-making processes can lead to slow and time-consuming decisions, causing delays and inefficiencies. The product owner may avoid making decisions that require involving others to prevent delays. Delays can also arise from indecisiveness or lack of clarity in the client organization's requirements. For mitigation of the software project issues at a large scale at the organizational level, researchers provide a theoretical framework to identify critical success factors and barriers for implementing software process improvement in large-scale software development organizations. The identification of these factors helps organizations understand the human-related factors that impact software process improvement and enables them to address organizational issues effectively [20]. We discovered some interesting parallels and differences when we compared our case study to the existing literature [11, 12, 29].

The most common causes of challenges identified in the literature were directly or indirectly related to communication, coordination, collaboration, requirements engineering, and dependencies issues in multiple teams. Requirements engineering issues lead to many more problems, such as poor usability of the final products [28]. In this study, case companies face similar agile scaling challenges reported in the literature, such as lack of communication and coordination [8], inter-dependency between teams [33], distributed teams [15] system integration [32], and quality assurance challenge[30]. In our case study, deadlines, continuous requirements, team dependencies, and scope challenges were linked to a need for more communication, collaboration, and interdependent teams. In addition to the common challenges with literature, we found some additional challenges related to testing and quality assurance, such as a lack of acceptance testing, ignored quality standards, and complex and unstructured codes. We also identified sprint-related issues such as unprioritized backlogs, pending user stories in the product backlog, poorly documented user stories, and blind sprints (under or overestimated sprints), where the proper estimation for the sprint was lacking. We also found a supplementary challenge in managing the workload and expertise in LSAD teams. We identified some additional challenges faced by LSAD organizations, such as hard-to-get feedback from clients (organization), organizational fragmentation, and lack of common structure and balancing technical and managerial responsibilities.

Quality assurance issues such as a lack of acceptance testing occur because testers attempt to test the system on behalf of the end users who cannot do so. The leading causes of unstructured code are missing quality standards, developers ignoring quality standards, new inexperienced developers, or developers under pressure to meet deadlines. Sprint-related challenges, such as un-prioritized product backlogs, are caused by the absence of formal prioritization techniques such as story points, etc., and teams' lack of interest, knowledge, or involvement also leads to un-prioritized backlogs. Pending user stories are caused by the developers continuously changing their context, i.e., frequently jumping from one task to another. The other reason for pending cases in the product backlog is associated with late information sharing among interdependent teams about the tasks. Further, this problem happens when the product owners impose new cases to be completed before the old ones are already in the backlog. Blind sprints are caused by short-sighted planning in large-scale agile software development projects; teams are not forced to make long-term plans as they are set for just sprint planning. If they do not plan for a longer time, issues such as blind sprints happen because of a lack of estimation, leaving teams confused about how much work to bring into the sprint. It leads to failure in splitting product backlog items into small, discrete tasks. Consequently, the team needs to be able to assess the workload appropriately. Several strategies are reported in [3] to handle similar challenges, but they require empirical validation in industrial settings.

While the challenge of workload and managing expertise in the LSAD teams is likely due to a shortage of experts within the team or a situation where only a few experienced team members are handling a significant portion of the work, leaving other team members with less involvement. This situation can have several effects on the team's productivity and morale. The workload imbalance can lead to uneven distribution of work, resulting in some team members being overworked while others not being fully utilized. This can result in decreased productivity and efficiency and increased stress levels for the team members. Moreover, lacking expertise can delay project timelines. To mitigate these effects, it is essential to ensure that teams have adequate experts and experienced members to handle project tasks effectively. This can be achieved by providing training and development opportunities to less experienced team members and ensuring the workload is evenly distributed[31]. Additionally, communication and coordination within the team need to be improved to avoid confusion and delays[23].

Organizational fragmentation and lack of a common structure can arise from several factors, such as decentralization, mergers and acquisitions, and geographical dispersion. Decentralization can lead to fragmented decision-making and a lack of standardization in processes and procedures, resulting in inconsistency and confusion. Geographical dispersion can cause communication and coordination challenges, making maintaining a cohesive organizational structure and culture difficult[17].

On the other hand, managing technical and managerial responsibilities can be challenging, especially for technical experts who become department managers. They may struggle to balance their technical duties with managerial responsibilities, leading to difficulties in delivering projects and man-

aging teams effectively. This can cause team difficulties and hamper productivity and innovation. It is crucial to address these challenges by providing proper training and support for managers to acquire the necessary managerial skills while maintaining their technical expertise[31]. Organizations can delegate responsibilities effectively and encourage team collaboration to mitigate the negative effects of managing technical and managerial responsibilities.

It would be interesting in the future to study some of the large-scale agile issues in the context of complex product and system development, as well as large-scale agile development challenges in the context of Non-Technical Debt.

6. CONCLUSION

In conclusion, this study investigated challenges in LSAD projects within two Swedish companies, encompassing LSAD processes and practices, LSAD teams, and organizational-level factors. The identified challenges in LSAD processes and practices involved complex system testing, including integration, regression, and user acceptance testing, resulting in "test debt" and compromised quality assurance. LSAD teams faced difficulties such as continuous changes, a lack of expertise, and challenges in workload management, communication, and coordination. At the organizational level, challenges included obtaining timely feedback, organizational fragmentation, conflicting project management approaches, and balancing technical and managerial responsibilities.
These findings align with existing literature on challenges related to communication, coordination, collaboration, requirements engineering, and team dependencies. Furthermore, additional challenges emerged in testing and quality assurance, workload and expertise management within LSAD teams, and organizational issues like obtaining feedback and addressing fragmentation.

To address these challenges, it is crucial to implement strategies that improve communication and coordination and foster interdependent teams. Involving experts and experienced members in LSAD projects can enhance decision making and problem-solving processes. Mitigating workload and expertise management challenges necessitates a balanced distribution of work among team members and providing training and development opportunities. Addressing organizational fragmentation and establishing a common structure require standardization and effective communication channels. Managing technical and managerial responsibilities can be improved through targeted training, ongoing support, and delegation of responsibilities based on individual strengths and expertise.

7. FUTURE WORK

In light of the case study findings, the following research directions should be examined in the future:

Strategies for Effective Test Management in LSAD: Further research can focus on developing and evaluating strategies to address the challenges of testing complex systems in large-scale agile projects. This can include exploring innovative approaches to integration, regression, and user acceptance testing and finding ways to mitigate "test debt" in LSAD.

Enhancing Team Dynamics and Expertise Management in LSAD: Research can explore methods to improve team stability, expertise management, and workload distribution in LSAD projects. This can involve studying the impact of continuous team changes on project timelines and team morale and finding ways to balance the number of experts and experienced team members to increase speed and agility.

Overcoming Organizational Challenges in LSAD: Further research can focus on addressing the organizational-level challenges identified, such as obtaining timely feedback from client organizations, resolving organizational fragmentation, and managing conflicts arising from conflicting project management approaches. Strategies to streamline decision making processes and improve clarity in client requirements can also be explored.

Exploring the Relationship between LSAD Challenges and Non-Technical Debt: Future research can investigate the connection between LSAD challenges and non-technical debt, such as social, process, and people-related issues. This can involve examining the impact of non-technical debt on LSAD project outcomes and identifying effective strategies to manage and mitigate non-technical debt in LSAD.

Studying LSAD Challenges in Complex Product and System Development: Research can focus on understanding the challenges faced in large-scale agile development projects involving complex products and systems. This can include exploring the unique requirements, engineering issues, coordination challenges, and dependencies inherent in such projects and identifying effective approaches to address them.

8. ACKNOWLEDGMENT

This work was supported by NODLA Project funded by the Knowledge Foundation Sweden.

9. REFERENCES

[1] P. Abrahamsson, O. Salo, J. Ronkainen, and J. Warsta. Agile software development methods: Review and analysis. *VTT publication 478*, 2017.

[2] M. O. Ahmad, D. Dennehy, K. Conboy, and M. Oivo. Kanban in software engineering: A systematic mapping study. *Journal of Systems and Software*, 137:96–113, 2018.

[3] M. O. Ahmad and T. Gustavsson. The pandora's box of social, process, and people debts in software engineering. *Journal of Software: Evolution and Process*, page e2516, 2022.

[4] M. O. Ahmad, P. Kuvaja, M. Oivo, and J. Markkula. Transition of software maintenance teams from scrum to kanban. In *2016 49th Hawaii International Conference on System Sciences (HICSS)*, pages 5427–5436. IEEE, 2016.

[5] K. M. Alpi and J. J. Evans. Distinguishing case study as a research method from case reports as a publication type. *Journal of the Medical Library Association: JMLA*, 107(1):1, 2019.

[6] M. Alqudah and R. Razali. A review of scaling agile methods in large software development. *International*

Journal on Advanced Science, Engineering and Information Technology, 6(6):828–837, 2016.

[7] K. Beck, M. Beedle, A. Van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, et al. Manifesto for agile software development. 2001.

[8] M. Berntzen, R. Hoda, N. B. Moe, and V. Stray. A taxonomy of inter-team coordination mechanisms in large-scale agile. *IEEE Transactions on Software Engineering*, 49(2):699–718, 2022.

[9] K. Conboy and N. Carroll. Implementing large-scale agile frameworks: challenges and recommendations. *Ieee Software*, 36(2):44–50, 2019.

[10] S. Das and K. Gary. Agile transformation at scale: A tertiary study. In *International Conference on Agile Software Development*, pages 3–11. Springer, 2021.

[11] K. Dikert, M. Paasivaara, and C. Lassenius. Challenges and success factors for large-scale agile transformations: A systematic literature review. *Journal of Systems and Software*, 119:87–108, 2016.

[12] T. Dingsøyr, T. E. Fægri, and J. Itkonen. What is large in large-scale? a taxonomy of scale for agile software development. In *International Conference on Product-Focused Software Process Improvement*, pages 273–276. Springer, 2014.

[13] T. Dybå and T. Dingsøyr. Empirical studies of agile software development: A systematic review. *Information and software technology*, 50(9-10):833–859, 2008.

[14] T. Dyba and T. Dingsoyr. What do we know about agile software development? *IEEE software*, 26(5):6–9, 2009.

[15] H. Edison, X. Wang, and K. Conboy. Comparing methods for large-scale agile software development: A systematic literature review. *IEEE Transactions on Software Engineering*, 2021.

[16] J. Fereday and E. Muir-Cochrane. Demonstrating rigor using thematic analysis: A hybrid approach of inductive and deductive coding and theme development. *International journal of qualitative methods*, 5(1):80–92, 2006.

[17] M. Gupta, A. Sharma, S. Hooda, and J. S. Bhatia. Distributed agile software development (dasd) process. *Agile Software Development: Trends, Challenges and Applications*, pages 187–203, 2023.

[18] B. Hobbs and Y. Petit. Agile methods on large projects in large organizations. *Project Management Journal*, 48(3):3–19, 2017.

[19] M. Kalenda, P. Hyna, and B. Rossi. Scaling agile in large organizations: Practices, challenges, and success factors. *Journal of Software: Evolution and Process*, 30(10):e1954, 2018.

[20] A. A. Khan, J. Keung, S. Hussain, M. Niazi, and M. M. I. Tamimy. Understanding software process improvement in global software development: a theoretical framework of human factors. *ACM SIGAPP Applied Computing Review*, 17(2):5–15, 2017.

[21] B. Kischelewski and J. Richter. Implementing large-scale agile-an analysis of challenges and success factors. 2020.

[22] M. Kowalczyk, B. Marcinkowski, and A. Przybyłek. Scaled agile framework. dealing with software process-related challenges of a financial group with the action research approach. *Journal of Software: Evolution and Process*, 34(6):e2455, 2022.

[23] P. Marnada, T. Raharjo, B. Hardian, and A. Prasetyo. Agile project management challenge in handling scope and change: A systematic literature review. *Procedia Computer Science*, 197:290–300, 2022.

[24] J. A. Maxwell et al. *Designing a qualitative study*, volume 2. The SAGE handbook of applied social research methods, 2008.

[25] N. B. Moe, B. H. Dahl, V. Stray, L. S. Karlsen, and S. Schjødt-Osmo. Team autonomy in large-scale agile. In *Proceedings of the Annual Hawaii International Conference on System Sciences (HICSS)*, pages 6997–7006. AIS Electronic Library, 2019.

[26] S. Nilsson Tengstrand, P. Tomaszewski, M. Borg, and R. Jabangwe. Challenges of adopting safe in the banking industry–a study two years after its introduction. In *International Conference on Agile Software Development*, pages 157–171. Springer, 2021.

[27] P. Runeson and M. Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical software engineering*, 14(2):131–164, 2009.

[28] H. Saeeda, F. Arif, and N. M. Minhas. Usability software engineering testing experimentation for android-based web applications: Usability engineering testing for online learning management system. In *Application Development and Design: Concepts, Methodologies, Tools, and Applications*, pages 397–415. IGI Global, 2018.

[29] H. Saeeda, J. Dong, Y. Wang, and M. A. Abid. A proposed framework for improved software requirements elicitation process in scrum: Implementation by a real-life norway-based it project. *Journal of Software: Evolution and Process*, 32(7):e2247, 2020.

[30] R. Shuib et al. Towards adopting software quality assurance in agile development methodology. *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, 12(3):2152–2157, 2021.

[31] J. Sithambaram, M. H. N. B. M. Nasir, and R. Ahmad. Issues and challenges impacting the successful management of agile-hybrid projects: A grounded theory approach. *International journal of project management*, 39(5):474–495, 2021.

[32] J.-P. Steghöfer, E. Knauss, J. Horkoff, and R. Wohlrab. Challenges of scaled agile for safety-critical systems. In *Product-Focused Software Process Improvement: 20th International Conference, PROFES 2019, Barcelona, Spain, November 27–29, 2019, Proceedings 20*, pages 350–366. Springer, 2019.

[33] V. Stray, N. B. Moe, and A. Aasheim. Dependency management in large-scale agile: a case study of devops teams. In *Proceedings of the Annual Hawaii International Conference on System Sciences (HICSS)*, pages 7007–7016. AIS Electronic Library, 2019.

ABOUT THE AUTHORS:

Hina Saeeda is doing Postdoc at Karlstad University, Sweden. She received her Masters's degree from NUST Pakistan and her Ph.D. from Ocean University of China. Previously, she worked as an Assistant Professor of software engineering at the SZABIST University of Pakistan. Her research interests include Agile software development, Usability engineering, socio-technical debt, Software requirements engineering, Software cost estimation, and software effort estimation.

Muhammad Ovais Ahmad is an Associate Professor at Karlstad University Sweden and Adj. Professor at the University of Oulu, Finland. He received his MSc. and Ph.D. from the University of Oulu, Finland. Previously, he worked as a Professor of software engineering at the Gdansk University of Technology, Poland, research fellow, and program coordinator of the European Masters Programme in Software Engineering at the University of Oulu, Finland. His research interests are evidence-based software engineering, Agility and software project management, and socio-technical debt.

Tomas Gustavsson is an Assistant Professor at Karlstad University and Chairman of the Swedish Project Academy. He has a Ph.D. in Information Systems from Karlstad University, an MBA from Stockholm University, and many years of industry experience as an IT and management consultant. He is also the author of several prize-winning textbooks on agile development methods. His main research interests are agile working methods, teamwork, coordination, and leadership in large-scale agile software development.